Phased Array Radar Exploration Workshop

Python Version

ADALM-PHASER

wiki.analog.com/phaser

ANALOG DEVICES

AHEAD OF WHAT'S POSSIBLE™

Please ask any questions at this forum: **ez.analog.com/adieducation/university-program**

Contents

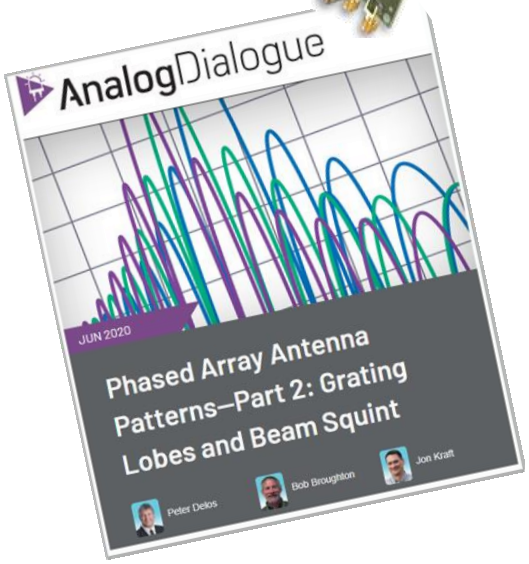
INTRODUCTION

1. Background and Purpose

Phased array beamforming has been used in radars and communication systems since the mid-20th century. But in recent years, these systems have seen greater adoption in areas such as 5G mobile communications, automotive radar, satellite communications, and military applications.

Yet, it can be difficult to gain an intuitive understanding of the fundamental concepts of these electronically steered arrays (ESA). Therefore, the purpose of this workshop is to demystify the phased array terminology and equip you with a basic understanding of its underlying principles. This will be accomplished with step by step, hands-on, labs covering the following topics:

1- Using software to control the array and process data
2- Setting the antenna's steering angle
3- Understanding the phased array's antenna pattern
4- Observing Antenna Impairments:
    a. Sidelobes and Tapering
    b. Grating Lobes
    c. Beam Squint
    d. Quantization Sidelobes
5- Analog, Digital, and Hybrid Beamforming
6- Monopulse Tracking
7- FMCW Phased Array Radar



2. Additional Information and Resources

1- More information on the hardware and software control can be found here: wiki.analog.com/phaser

2- Much of this presentation and resultant lab exercises were taken from this article series: https://www.analog.com/en/analog-dialogue/articles/phased-array-antenna-patterns-part1.html

3- More information on Pluto and its software control using MATLAB can be found here: wiki.analog.com/sdrseminars

4- MATLAB versions of these labs can be found here: https://wiki.analog.com/phaser_matlab
    and
    https://github.com/mathworks/Phaser-Control-with-MATLAB

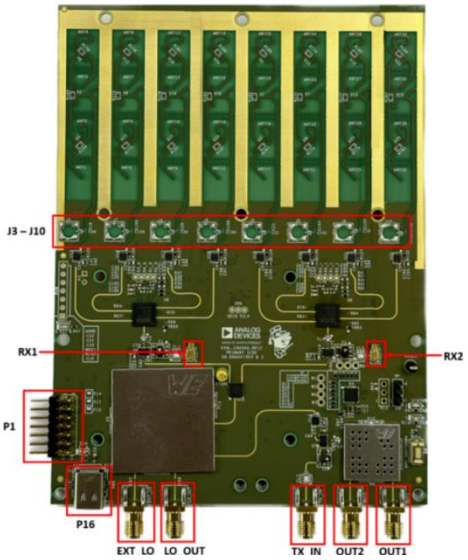
3. Lab Setup

To accomplish these labs, we will be using Analog Device's ADALM-PHASER (wiki.analog.com/phaser).

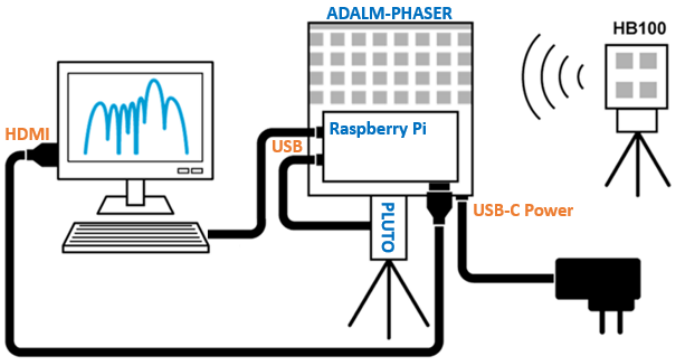
The Phaser has the Pluto and Raspberry Pi directly attached to it. The Raspberry Pi is used to convert commands from the laptop to SPI commands for the RF chips.

The connections necessary are:

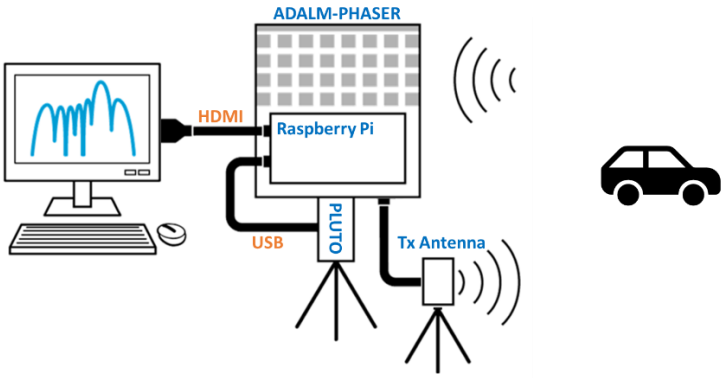
1. 5V, 3A USB-C power supply to the bottom USBC input on Phaser (labeled "P16")
2. HDMI cable from the Rasp Pi to a monitor
3. Center micro USB of Pluto connected to the Rasp Pi
4. For the radar labs: connect the transmit antenna to "OUT2"

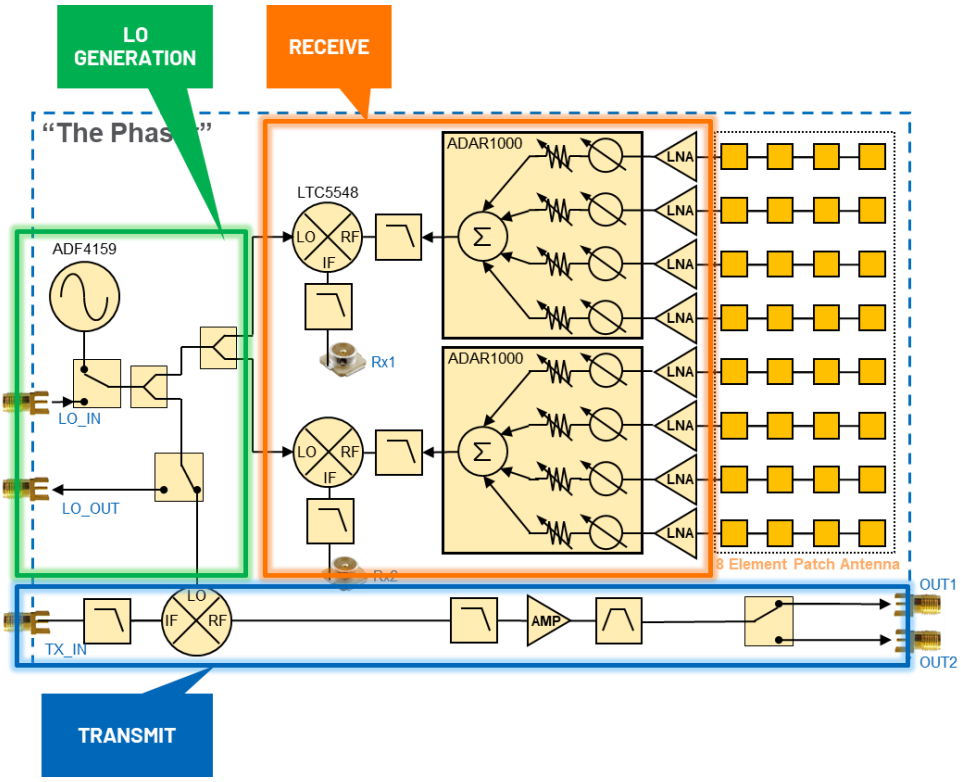


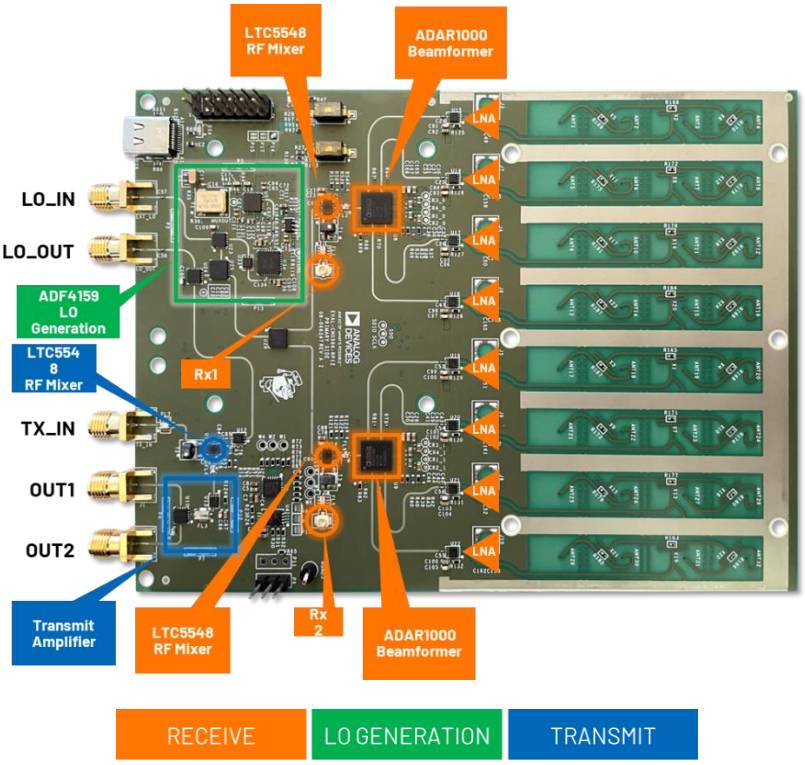
Beamforming Lab Setup



Radar Lab Setup

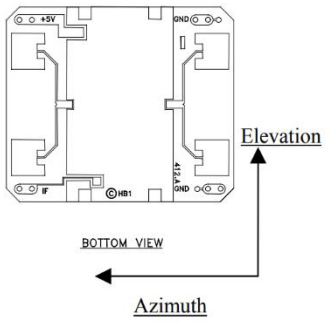


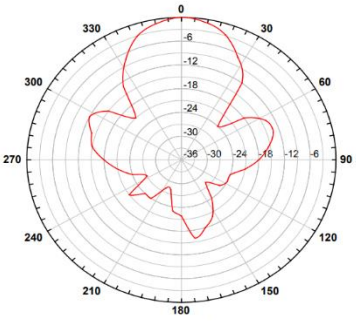


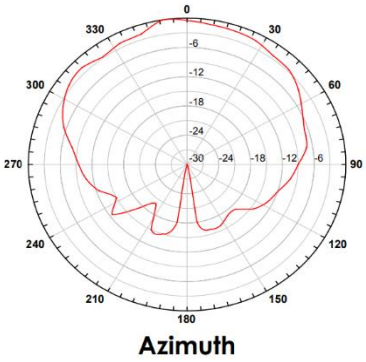


To generate the RF source for all of the radar labs, we will use the Phaser's Tx port. But for the phased arrays labs, it will be more convenient to use the battery powered HB100. The "HB100" is a low cost 10.5 GHz source commonly

used in occupancy detection systems. The version we use is the SEN0192 from DF Robot: https://www.dfrobot.com/product-1403.html.







https://www.limpkin.fr/public/HB100/HB100_Microwave_Sensor_Module_Datasheet.pdf

Using the HB100 allows us to have complete freedom in moving around the X band RF source. This will be very useful in the Monopulse Tracking lab where we can move the RF source around the room and observe the lock and tracking.

SDR AND SOFTWARE CONTROL

1. Training Objective

In this lab, we will learn how to control the Pluto SDR (software defined radio).

We will aim the HB100 (approx. 10.5 GHz) at the Phaser board. The Phaser will mix down that 10.5 GHz signal to about 2.2 GHz. We then digitize that 2.2 GHz signal and plot the time and frequency spectrum. By doing this we can see exactly what our HB100 frequency is (it is not well controlled – it could be anywhere from 10.1 to 10.7 GHz!). And we can also see if there are any other signals nearby our HB100's frequency.

Instructions

1- Aim the HB100 at the Phaser

2- Open Thonny, by choosing Start → Programming → Thonny

3- Select the "phaser_minimal_example.py" tab

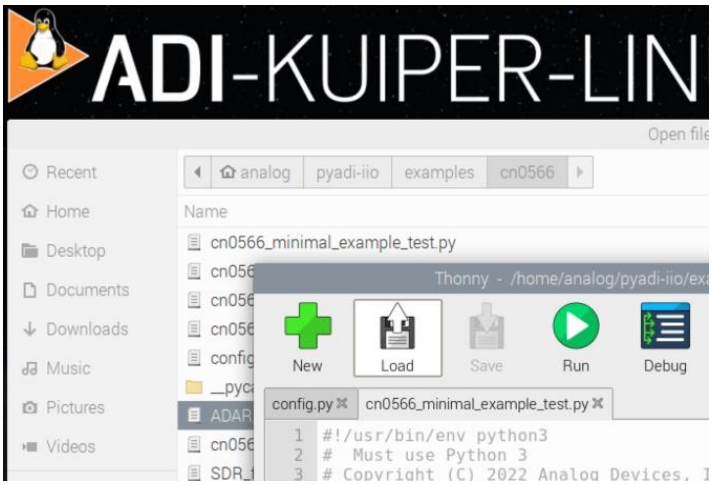


Figure 1: Run CN0566_minimal_example_test.py

4- Click "Run"

5- View the resultant graph:

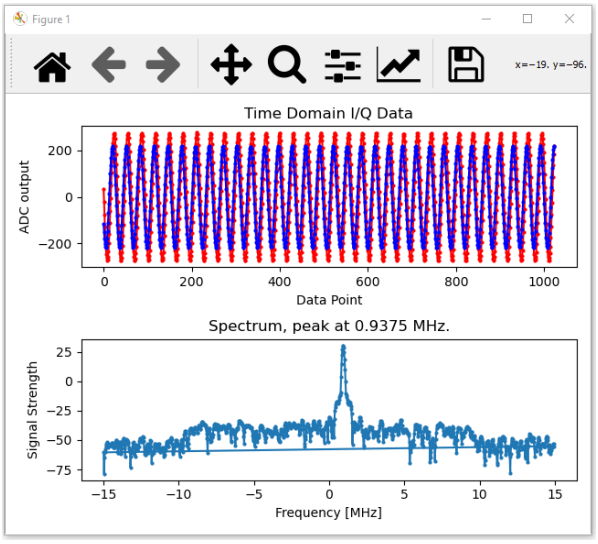


Figure 2: Plotting the Receive Signal of the HB100

Understand the Script

Let's understand that Python script we just ran, and make some modifications to it

Here's what we've done:

- Received the 10.525 GHz signal
- Downconverted it to 2.2 GHz
- Received the 2.2 GHz IF with the Pluto SDR
- Set the Pluto's internal PLLs to 2.2 GHz minus a small offset
- Set the Pluto's ADC sample rate to 30 MSPS
- Loaded a 20 MHz wide digital filter into the Pluto
- Capture a buffer of 1024 samples
- Plot the time domain samples
- Take the FFT of the samples, then plot.

So what does that Python script do?? The python script, "phaser_minimal_example.py" first takes care of some housekeeping operations - set the antenna to zero phase, equal gain on all elements, and set a few parameters in the Pluto SDR. Then we are simply plotting the buffers of data from Pluto.

Modify and Re-Run the script

1. Change line 176 to something between -10e6 and +10e6 (your choice!)

```
176  offset = 1000000 # add a small offset
177  my_cn0566.frequency = (
178      int(my_cn0566.SignalFreq + my_sdr.rx_lo - offset)
179  ) // 4  # PLL feedback is from the VCO's /4 output
```

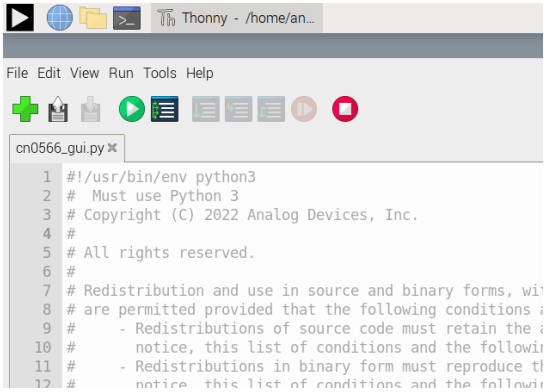
STEERING ANGLE

1. Training Objective

In this lab, we'll explore the relationship between the element to element phase shift and the resulting electrical steering angle

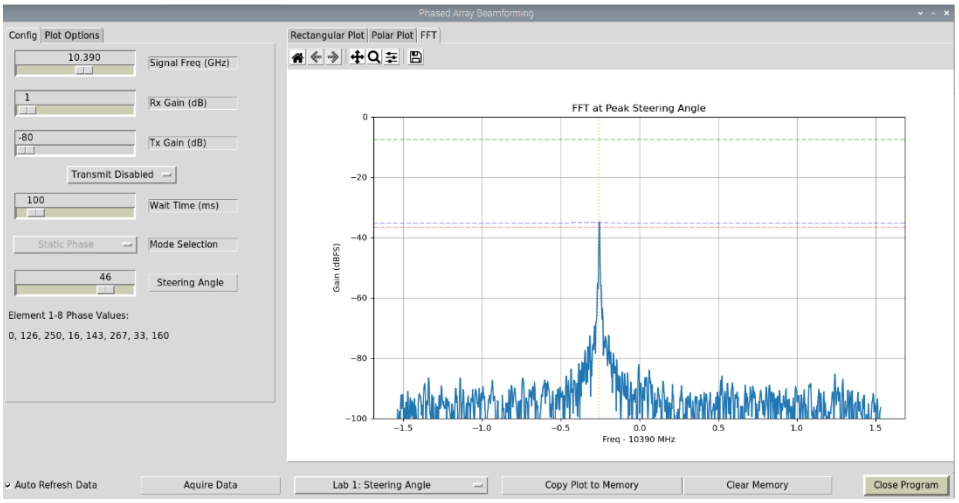
Instructions

1- Find the "phaser_gui.py" tab



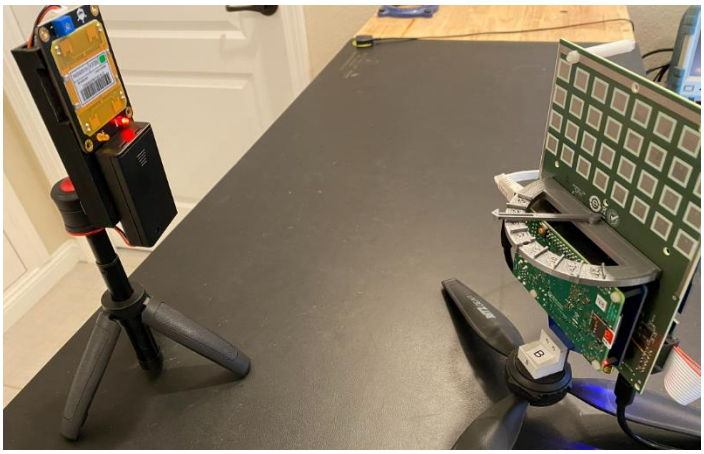
2- Press the green "Run" button

3- You'll see the FFT (amplitude vs frequency) of the HB100 source as received by the Phaser's array:



4- By adjusting the "Steering Angle" slider bar, you can change the phase values of each element.

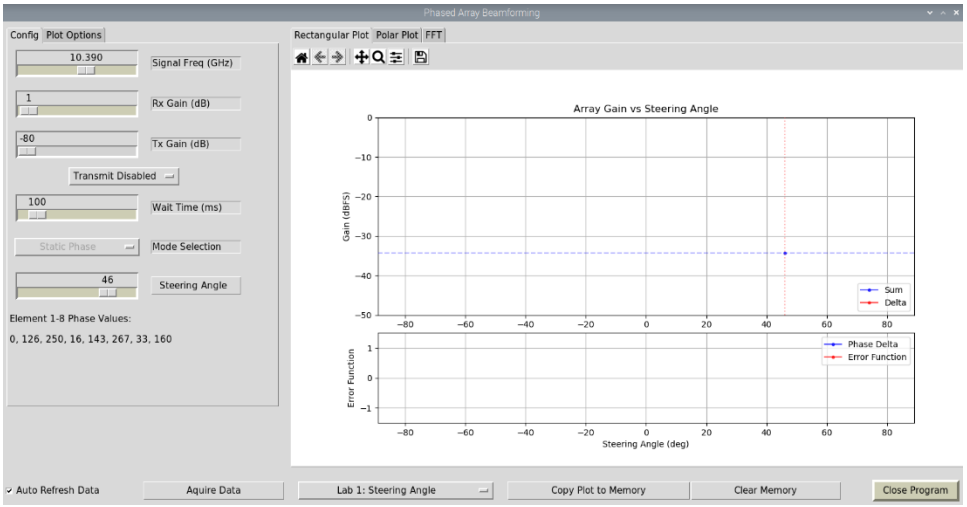
5- Move the HB100 to an angle of about 30 deg. An optional protractor can help you point this somewhat accurately. Just place it on top of the Raspberry Pi, and move the arrow to 30 deg.



6- Now slide the “Steering Angle” to find the phase delta that produces the maximum FFT amplitude.

7- What phase delta do you observe that produces the maximum FFT amplitude?

8- Now Click on the “Rectangular Plot” tab



9- This plots the peak FFT amplitude vs the selected steering angle

10- Move the Steering Angle slider bar again.

11- Does the amplitude move in a predictable way? What do you think is happening?

ARRAY FACTOR AND BEAMWIDTH

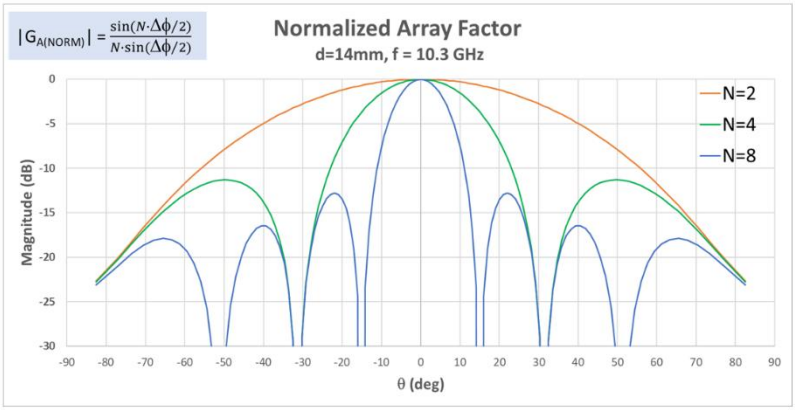
1- Background

In this lab, we will directly observe the array factor equation. And then observe how the beamwidth changes with steering angle.

Recall that the array factor of a uniform, equally weighted (constant amplitude), linear array is given by:

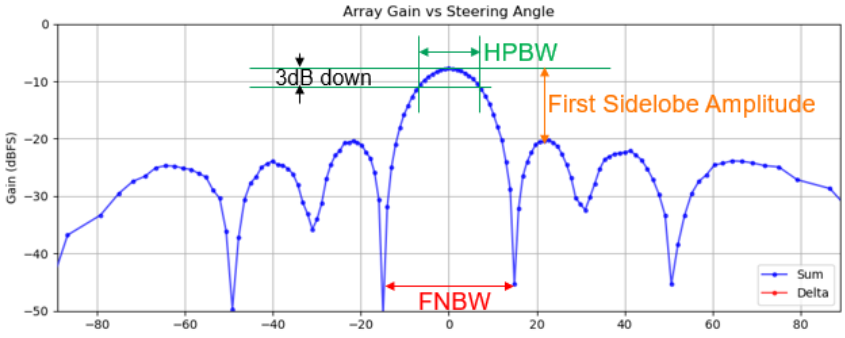
$$|G_{A(NORM)}| = \frac{\sin(N \cdot \Delta\phi/2)}{N \cdot \sin(\Delta\phi/2)}$$

And plotting this for various numbers of array elements (N) gives:



From this, we can make some measurements on the array:

1- Halfpower Beam Width (HPBW): Main lobe beamwidth, measured 3dB down from peak
2- First Null Beam Width (FNBW): Spacing between main lobe nulls
3- First Sidelobe Amplitude: Difference in amplitude (measured in dBc) from the main lobe to the first sidelobe on either side of the main lobe.
4- Peak Amplitude: signal strength of the main lobe (measured in dBFS for our lab)



From the array factor equation, with a frequency of 10.3GHz and element to element (d) spacing of 14mm, we can calculate the HPBW and FNBW for various numbers of elements:

	HPBW	FNBW
N=8	13°	30°
N=4	27°	62°
N=2	62°	180°

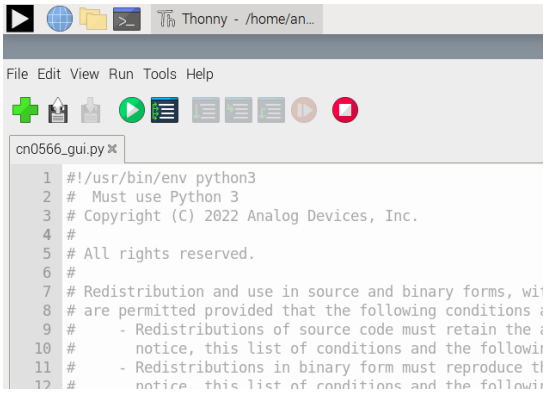
Let's measure this now and see how close we are to these calculated values.

2- Instructions with Phaser Hardware

1- First, **turn on the HB100 transmitter** and **place about 1 m away from the array**. The HB100 must be at the **mechanical boresight position** (i.e. broadside to the array).

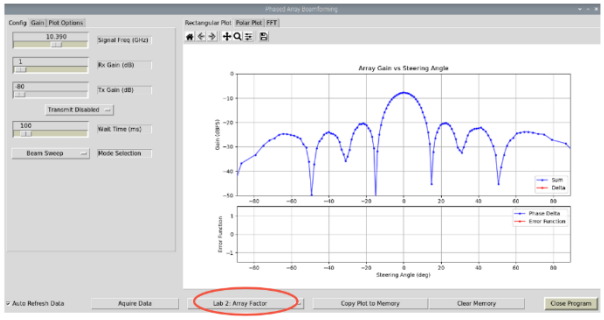
2- Open Thonny, by choosing Start → Programming → Thonny

3- Select "cn0566_gui_null_steering.py" tab



4- Click "Run"

5- In the Phaser GUI, select "Lab 2: Array Factor"



6- Slowly move the HB100 in a half-circle around the array and observe the changes

7- Does the main lobe's beamwidth remain constant as you move the RF source?

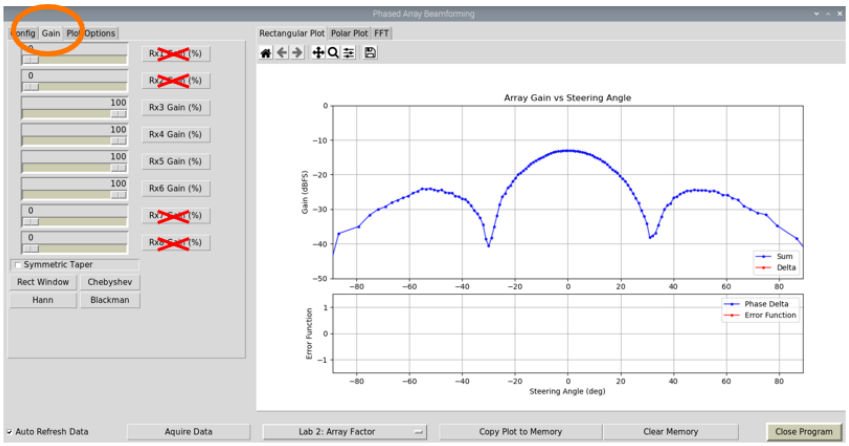
8- Return the HB100 to the mechanical boresight position (i.e. directly facing the array) and record the following:

N	Peak Amplitude (dBFS)	HPBW (°) Measured	HPBW (°) Calculated	FNBW (°) Measured	FNBW (°) Calculated	First Sidelobe Amplitude (dBc)
8			13		30	

a. For making the measurements, you can hover the cursor over the data point and record its value.

9- How do the HPBW and FNBW values compare with the calculated values?

10- In the Phaser GUI, select the "Gain" tab
11- Click the Rx1_Gain button to disable that channel.
12- Do the same for Rx2, Rx7, and Rx8. We now have a 4 element array!



13- Repeat the beamwidth measurements and compare to the calculated values and to the N=8 values.

N	Peak Amplitude (dBFS)	HPBW (°) Measured	HPBW (°) Calculated	FNBW (°) Measured	FNBW (°) Calculated	First Sidelobe Amplitude (dBc)
8			13		30	
4			27		62	
2			62		180	

MEASURING THE ACTUAL ANTENNA PATTERN

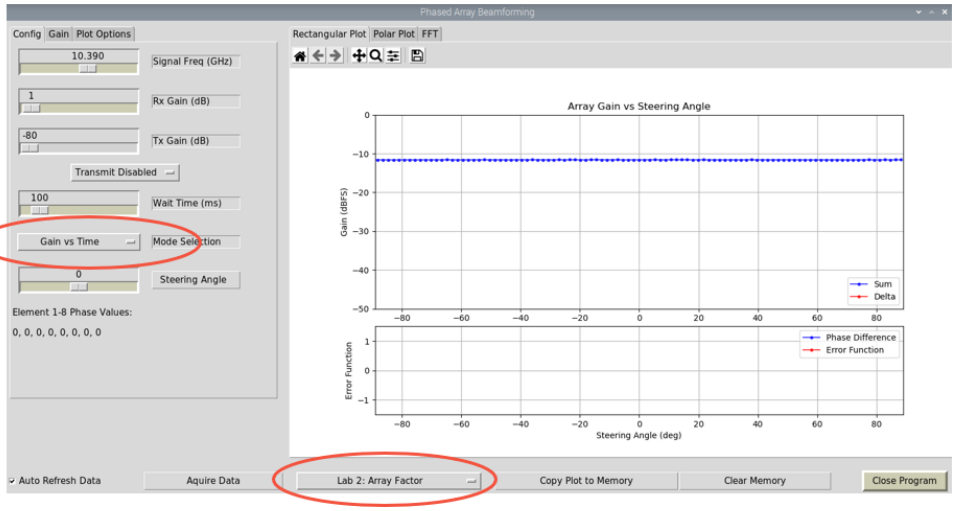
3- Background

In this lab, we'll make a simple measurement like what you would do in an antenna chamber. It certainly won't be perfect, but you'll experience the process and we'll measure the sidelobe levels and see how they compare to the electrical scan method.

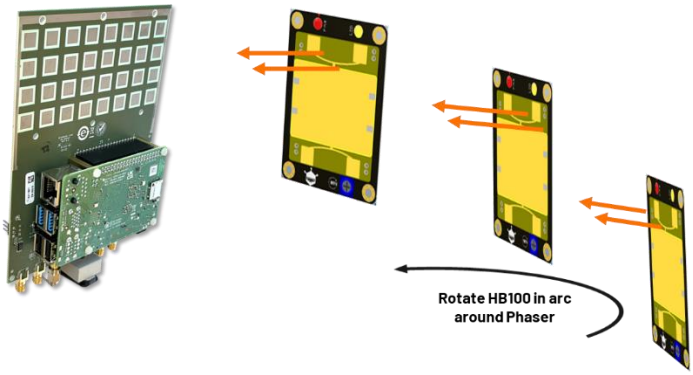
4- Instructions

5- In the Phaser GUI, select (again) the "Lab 2: Array Factor" from the drop down menu.

6- In the "Config" tab, select "Signal vs Time" from "Mode Selection". This plots the peak amplitude vs time.

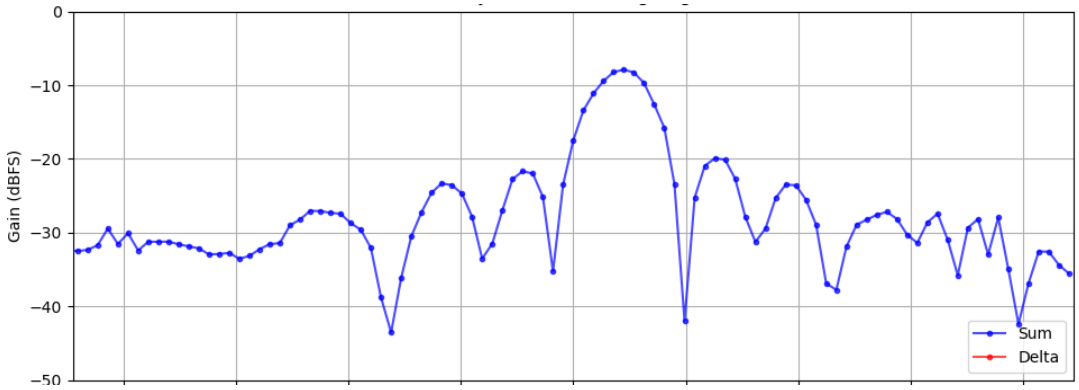


7- Now rotate the HB100 in a radius around the Phaser board –keep the HB100 pointed at Phaser!



8- Start at the left position (-90 deg), and move around to the right position (+90 deg). Keep a smooth, consistent speed!

9- Practice a few times, then try time it so that one smooth rotation covers the entire graph span. After a few practice runs, you may get something that looks like this:



10- Ok, so we can't really get angular measurements from this (we can never rotate it perfectly). But we can get accurate lobe amplitudes. Compare these amplitudes to your earlier measurements. How close are they?

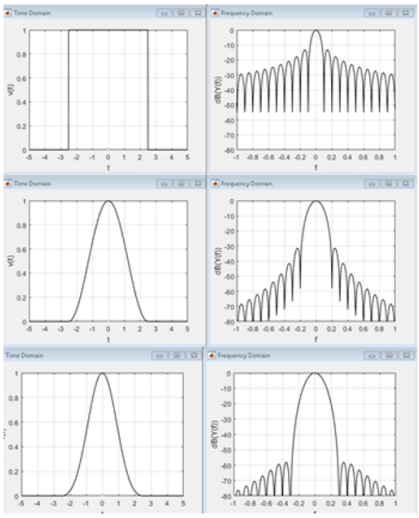
11- Repeat the process, but change the "Steering Angle" from 0 deg to 30 deg.

SIDELOBES AND TAPERING

1- Background

In this lab, we'll observe the side lobe amplitudes as we reduce the gain of the antenna elements at the edge of the array.

Recall the impact to the sidelobe gain when we "window" or "taper" an array:



Boxcar – 1st sidelobe @ -13dBc Narrowest main lobe

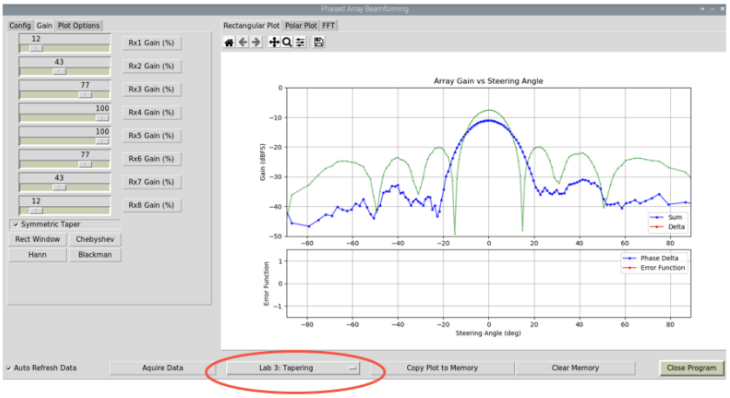
Hanning – 1st sidelobe < -30dBc Main lobe broadens

Blackman – Lowest sidelobes
Broadest main lobe

Note: windowing losses not shown in these examples

2- Instructions

1- In the Phaser GUI, select "Lab 3: Tapering"



2- Press "Copy Plot A", then try one of the tapering profile buttons.

3- What is the impact to sidelobe level, beamwidth, and peak gain?

4- Select "Symmetric Taper" and invent your own profile! Can you make a "better" taper?

GRATING LOBES

1- Background

In this lab, we will vary the effective element to element spacing to observe the formation of grating lobes. Then compare to our calculated values.

Consider the mechanical broadside condition (i.e. steering angle = 0 deg), then the main lobe position simplifies to:

$$\theta_{MAIN} = \sin^{-1}(m\,\lambda/d), \text{ for } m=0, \pm1, \pm2, \text{ etc.}$$

So if λ =29mm (which is the wavelength for 10.3 GHz), and d=14mm (which is indeed the element to element spacing on the Phaser array), then there is only one real solution to the equation above. And θ_{MAIN} = 0°. So no surprises there!

But if we change "d" to 42mm, then we will see 3 main lobes! And they will be located at:

$\theta = \sin^{-1}(m * 29mm/42mm) = 0°$ **and** ±44°

The true main lobe is at 0°. And then the ±44° are the grating lobes. And we'll actually see those grating lobes when we do the lab below.

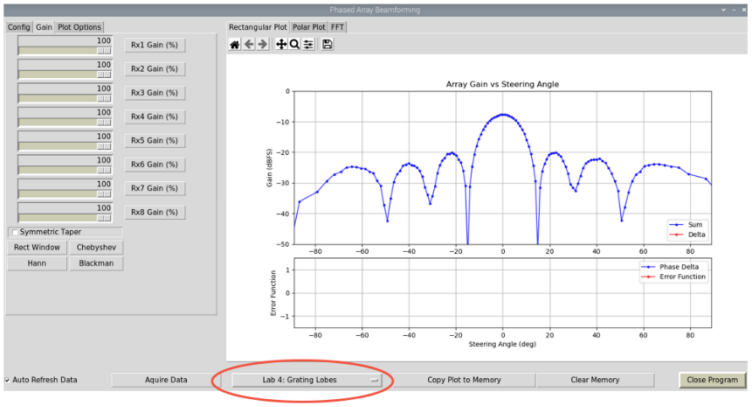
But we can also change "d" to 56mm. And in that case we will see "main" lobes at:

$\theta = \sin^{-1}(m * 29mm/56mm) = 0°$ **and** ±31° **and** ±90°

So let's try it out in the lab, and see those grating lobes directly.

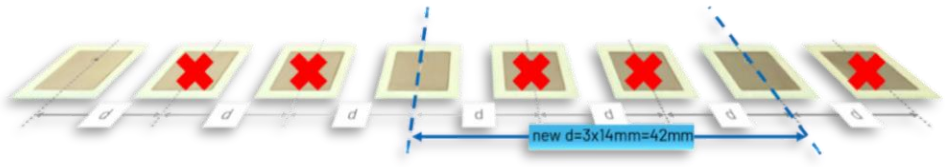
2- Instructions

3- In the Phaser GUI, select "Lab 4: Grating Lobes"

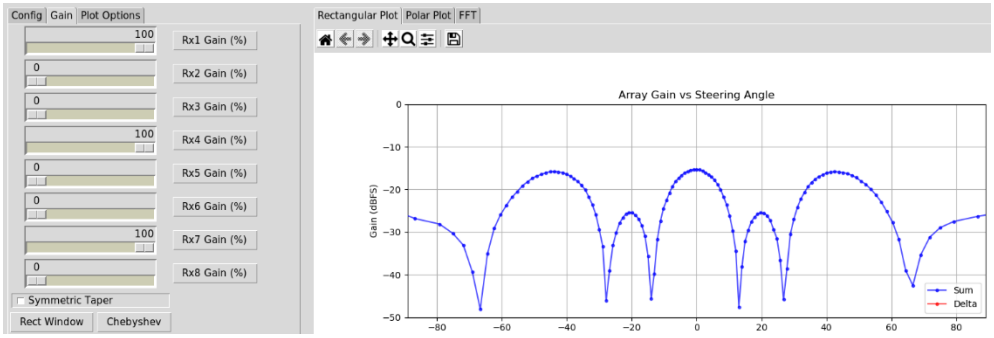


4- Set the RF source (HB100) to be directly in front of the array (full broadside).

5- In the gain tab, set Rx2, Rx3, Rx5, Rx6, and Rx8 to 0. Now our d = 3 * 14 mm = 42 mm

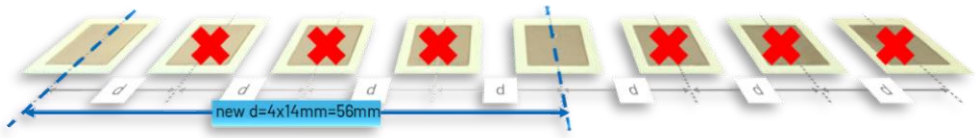


6- Do you see two additional “main” lobes? Does their peak angle match our calculations? Why are they broader than the true main lobe?

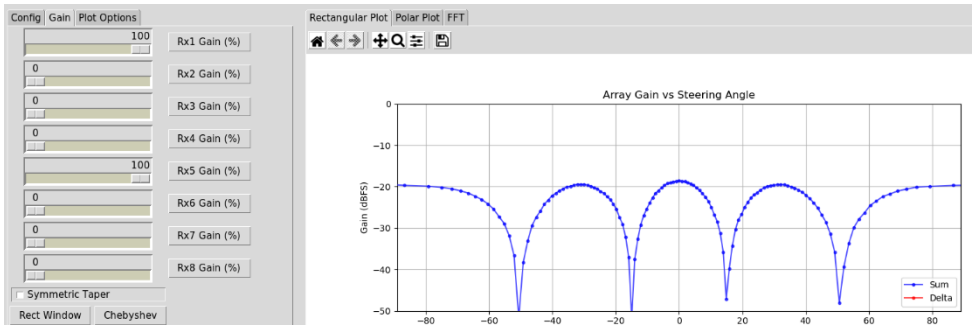


7- Do you see two additional “main” lobes? Does their peak angle match our calculations? Why are they broader than the true main lobe? Why do they have the same amplitude of the main lobe?

8- Let’s try it again, but now for d=56mm. Set Rx2, Rx3, Rx4, Rx6, Rx7, and Rx8 to 0.
Now our d = 4 * 14 mm = 56 mm



9- Again, check where the grating lobes are, against what we calculated previously:



BEAM SQUINT

1- Background

In this lab, we will observe the change in steering angle as a function of signal frequency

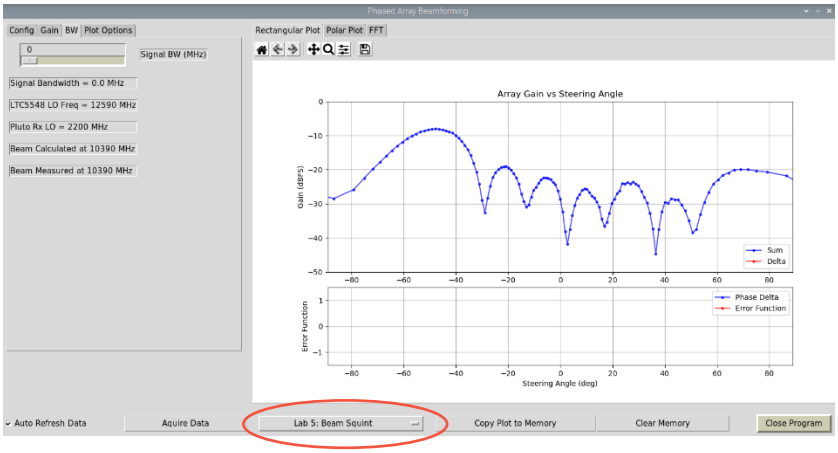
Recall that beam deviation (beam squint) vs frequency can be calculated as:

$$\Delta\theta = arcsin\left(\frac{f_0}{f} sin\,\theta_0\right) - \theta_0$$

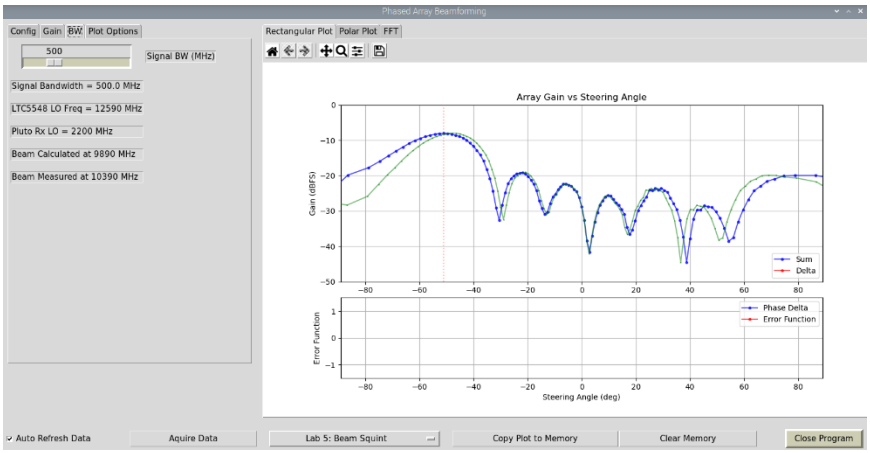
- For example
  - Let's set our carrier frequency to be 10.5 GHz, and f_0 = 10 GHz (500 MHz of BW)
  - We want to steer the beam to +/- 45° from mechanical boresight
  - $\Delta\theta$ = arcsin(10.5/10 * sin(45°)) - 45°= 3°
  - The beam will shift 3° at 10.5 GHz vs 10 GHz

2- Instructions

1- In the Phaser GUI, select "Lab 5: Beam Squint"



2- Set the RF source (HB100) to an angle of about 50 deg
3- Click "Copy Plot A" to copy the current data into memory
4- Record the peak angle (you can also turn on "Show Peak Angle" under "Plot Options" tab)
5- Change the "Signal BW" slider bar to 500 MHz



6- Record the new peak angle. Does the difference between the two peaks match our ~3° calculation?
7- Try other signal bandwidths and observe the effect.
8- Try other steering angles and observe the effect

Note: since our HB100 frequency source is fixed, we instead change the frequency at which the steering angle is calculated. i..e the "Beam Calculated at" frequency. Both methods are equivalent. Its just easier to change the calculated frequency in our lab. It also avoids other changes in the antenna pattern that would come from a new frequency. And that lets us do a more straightforward comparison.

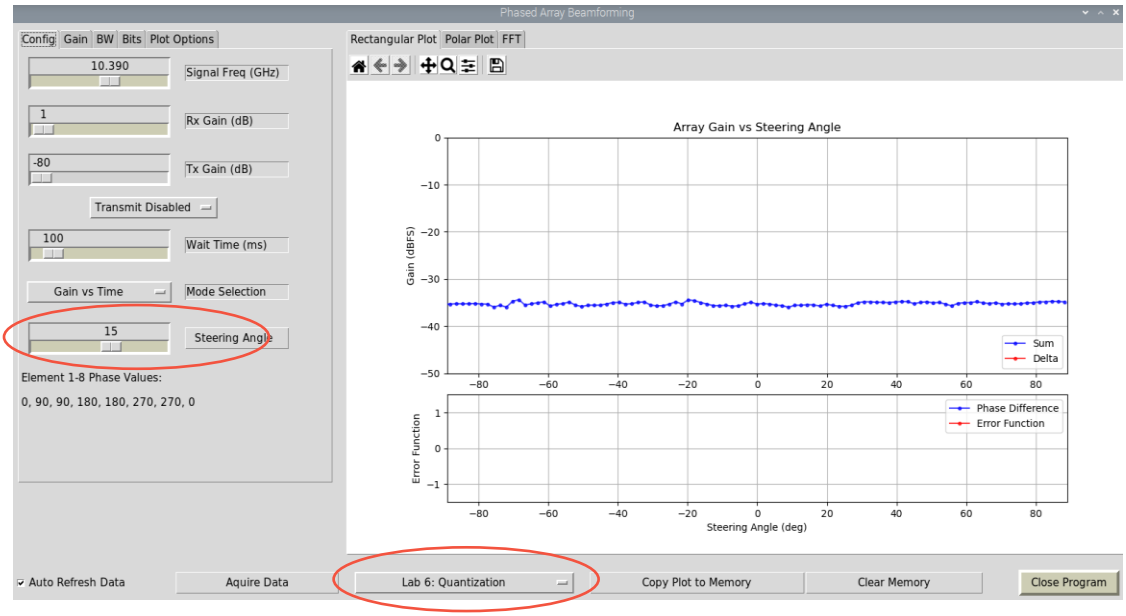
QUANTIZATION SIDELOBES

1- Background

In this lab, we will change the phase step size and observe the formation of quantization sidelobes

2- Instructions

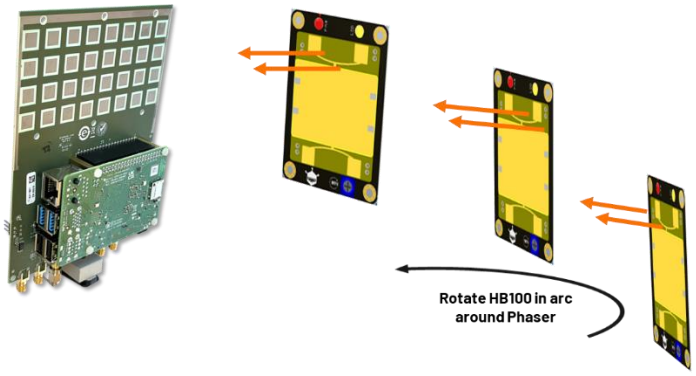
1- In the Phaser GUI, select "Lab 6: Quantization"



2- Click on the Gain tab and select a taper—we don't want any sidelobes! (Blackman is the pre-programmed default for this lab)

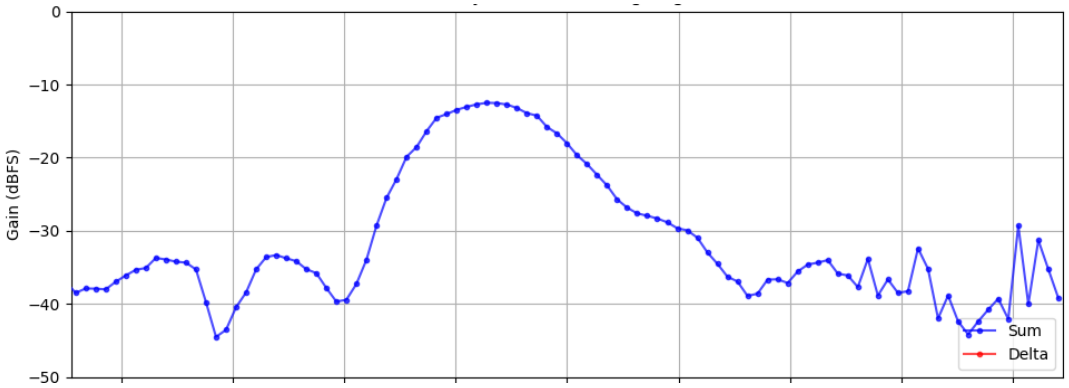
3- In the "Config" tab, set the steering angle to 15 deg

4- Move the HB100 in an arc around the Phaser.



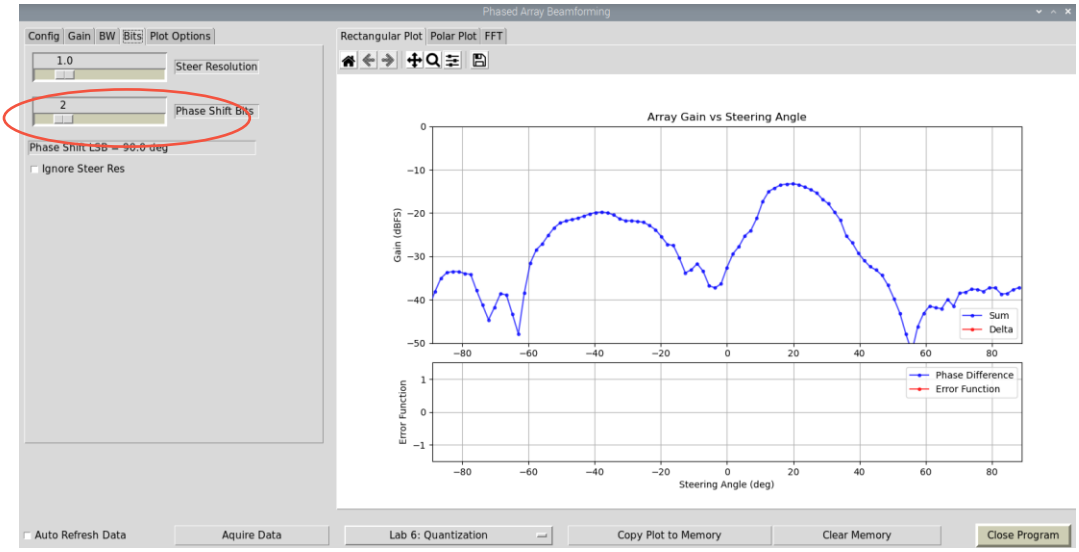
5- Keep the HB100 pointed at Phaser, and move at a smooth/consistent speed

6- With practice, it may look like this:



7- Our Blackman taper should have suppressed all the sidelobes. So we are just seeing the true mainlobe at 15°

8- Now, in the "Bits" tab: slide "Phase Shift Bits" to 2



9- Repeat moving the HB100 in an arc.

10- Do you see any new sidelobes?

PHASED ARRAY NULL STEERING

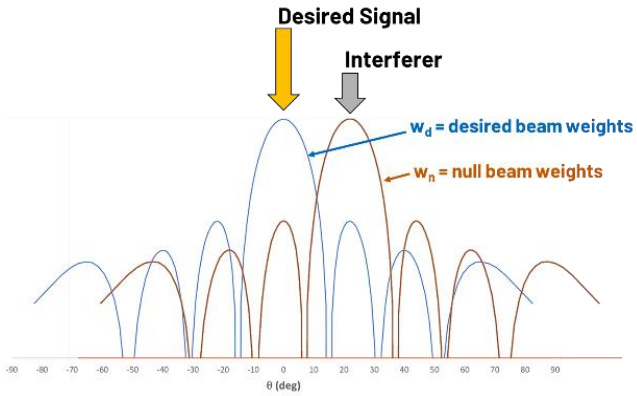
1- Background

Sometimes it may be desirable to reduce the antenna pattern in a certain direction. For example, if there is a known emitter in that direction that is interfering with the systems ability to monitor a desired signal. In this lab, we look at a simple approach for inserting a null into the antenna pattern in a certain direction.

This null steering example follows the excellent guidance found here:

https://www.mathworks.com/help/phased/ug/array-pattern-synthesis.html

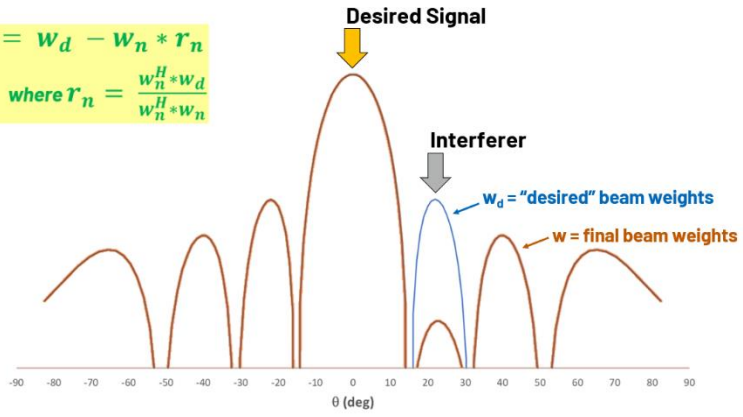
The goal of null steering is to change the beam pattern (antenna weights) to force a null at a specific location. In the diagram below, the desired signal is at 0 deg (mechanical boresight) and an interfere is at +22 deg (which is the location of the first sidelobe):



To place a null at the interferer, we first form the beam as we normally would. This give us our desired beamweights (wd). Then we form another beam where we want a null, giving the null beamweights (wn). Then we subtact the two giving the new (null steered) beamweights of:

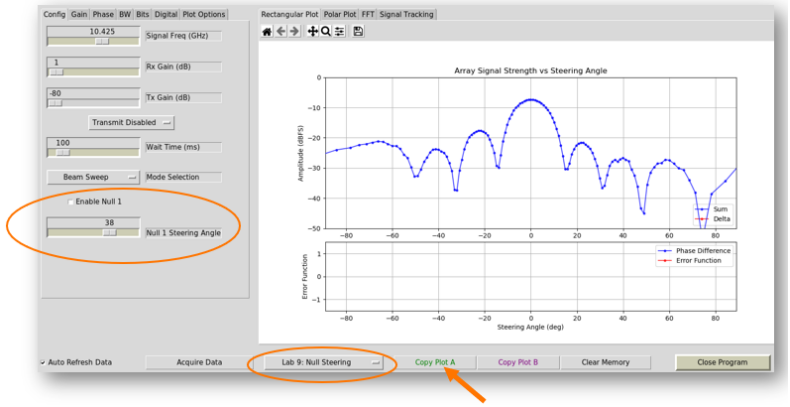
$$w = w_d - w_n * r_n$$

$$\text{where } r_n = \frac{w_n^H * w_d}{w_n^H * w_n}$$



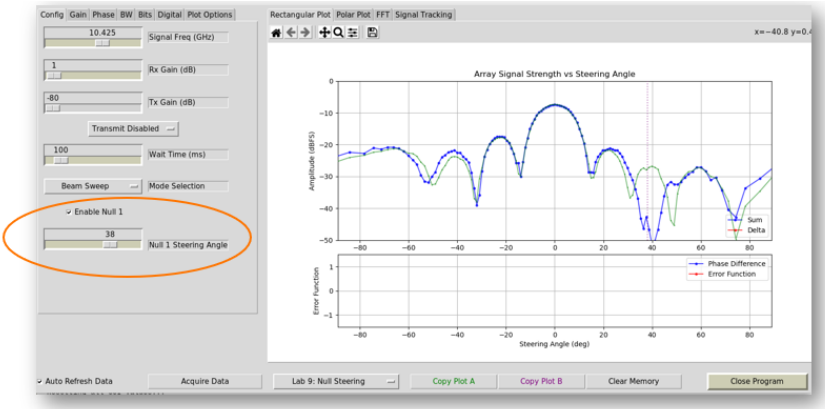
2- Instructions

1- In the Phaser GUI, select “Lab 9: Null Steering”
2- Click “Copy Plot A” to copy the current data into memory



3- Place a null on one of the sidelobes, and Click “Enable Null 1”

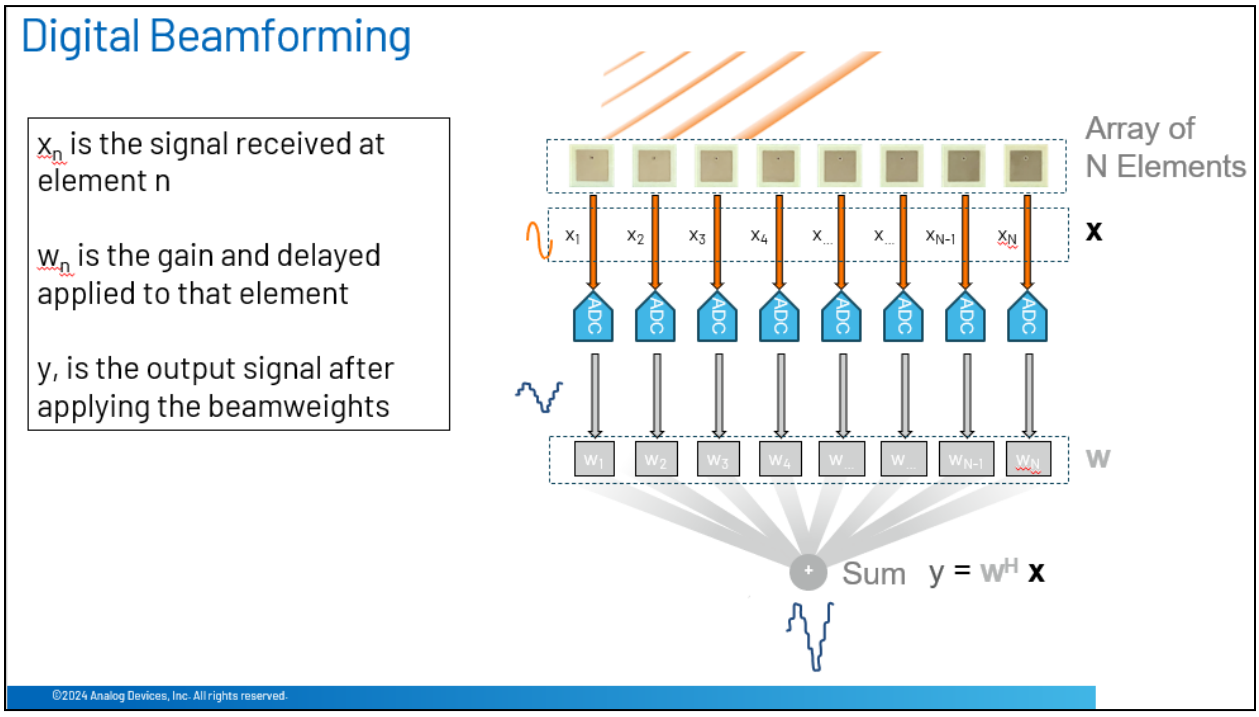
4- Rotate the HB100 around and note how the null is always at that location in the pattern



INTRO TO ADAPTIVE BEAMFORMING

1- Background

Adaptive beamforming allows us to continually adjust the beamweights to optimize the beam pattern in the presence of jammers and other interferes.



We want optimal beamweights such that:

$$\mathbf{y} = \mathbf{w}^{H}\mathbf{X} \rightarrow \mathbf{y} = \boldsymbol{w}_{\boldsymbol{opt}}^{\boldsymbol{H}}\mathbf{X}$$

So how do we find $\mathbf{w}_{opt}$????

Objective: Minimize output power of the beamformer without disturbing the SOI from a particular direction(θ_0):

$\boldsymbol{w}^{\boldsymbol{H}}\boldsymbol{R}\boldsymbol{w}$ is the output power (or "variance")
R is the covariance matrix of the received signals
s(θ_0) is the steering vector towards the desired signal direction θ_0

Solving this constrained optimization problem yields the following solution, called MVDR ("Minimum Variance Distortionless Beamformer"):

$$\boldsymbol{w}_{mvdr} = \frac{\boldsymbol{R}^{-1}\boldsymbol{s}}{\boldsymbol{s}^{H}\boldsymbol{R}^{-1}\boldsymbol{s}}$$

where:

s is the steering vector

R is the covariance matrix, size N x N

$\widehat{\boldsymbol{R}}$ is the covariance matrix estimate, size N x N, and found using the received samples

$$\widehat{\boldsymbol{R}} = \frac{1}{K}\sum_{k=1}^{K}\boldsymbol{x}_k\boldsymbol{x}_k^{H} = \frac{1}{K}\boldsymbol{X}\boldsymbol{X}^{H}$$

The weights $\boldsymbol{w}_{mvdr}$ can then be used just like we applied the delay-and-sum weights

$$\mathbf{y} = \boldsymbol{w}_{mvdr}^{H}\,\mathbf{X}$$

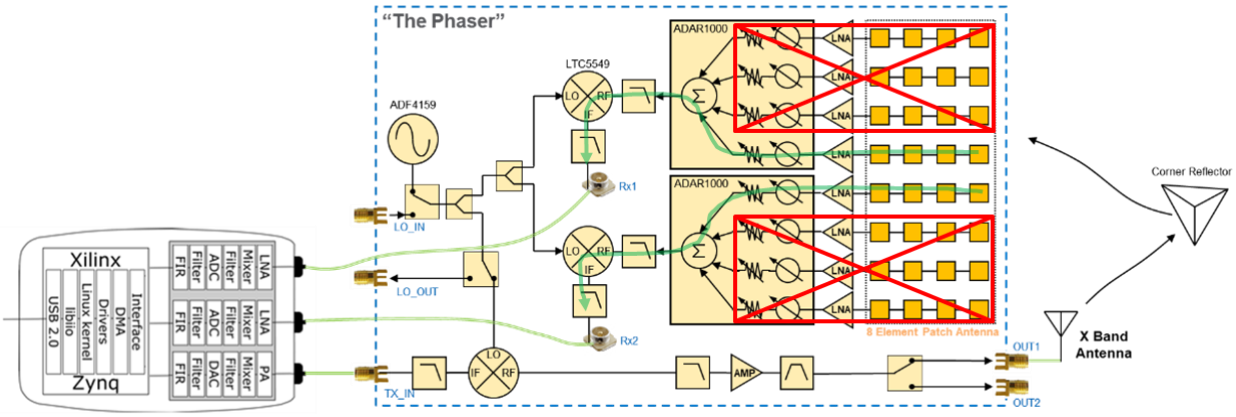
3- Instructions

1- For MVDR, we need digital channels. Analog channels (i.e. summing signals in the RF domain), will not give us the information we need to form the covariance matrix and pick the optimal beamweights.

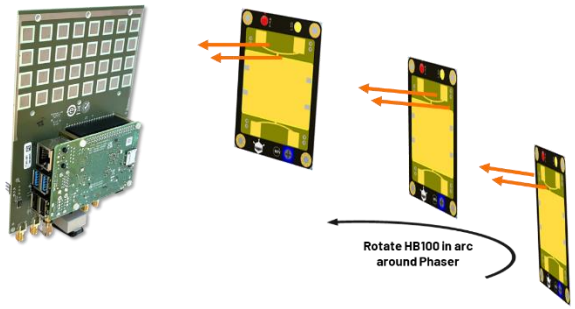
2- Phaser has 2 digital channels – with each digital channel receiving the summation of 4 antenna elements.

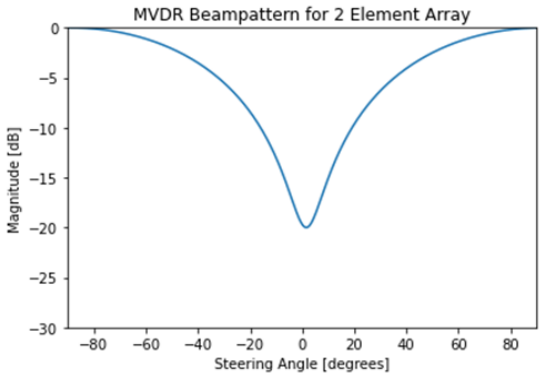
3- Therefore, we will transform Phaser from an 8 element hybrid beamformer into a 2 element digital beamformer!

   a. Simply turn off (attenuate) 3 of the 4 elements, and then only 2 elements will be left. And each will have its own digital receiver on Pluto



4- Close the GUI, in Thonny, if it is running.
5- Download the file “Phaser_MVDR.py” from https://github.com/jonkraft/PhaserExamples/tree/main/PythonExamples
6- Open the file “Phaser_MVDR.py” and click run.
7- Move the HB100 and observe how the 2 element beampattern changes.
    a. Where is null always being placed? Why?





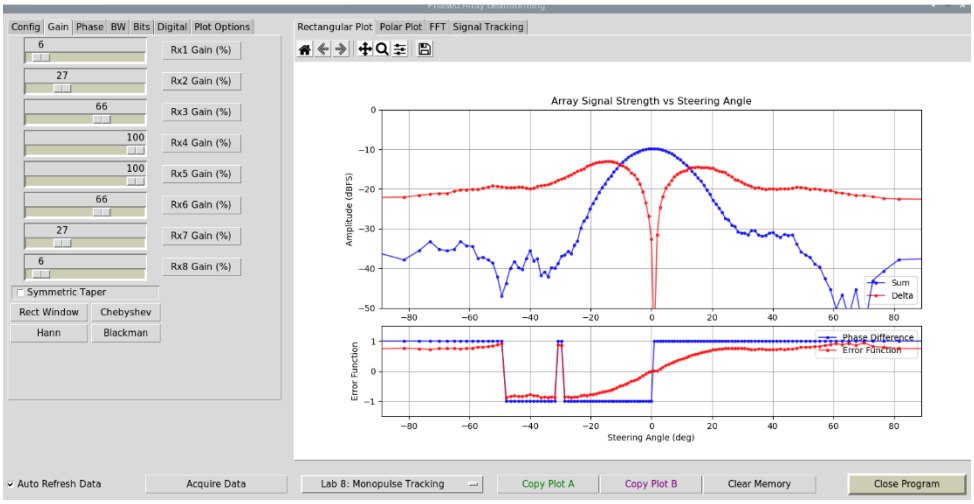
MONOPULSE TRACKING

1- Background

In this lab, we will implement the monopulse tracking function that we worked out in the previous lecture. Then we will observe it lock into a target and track its angle.

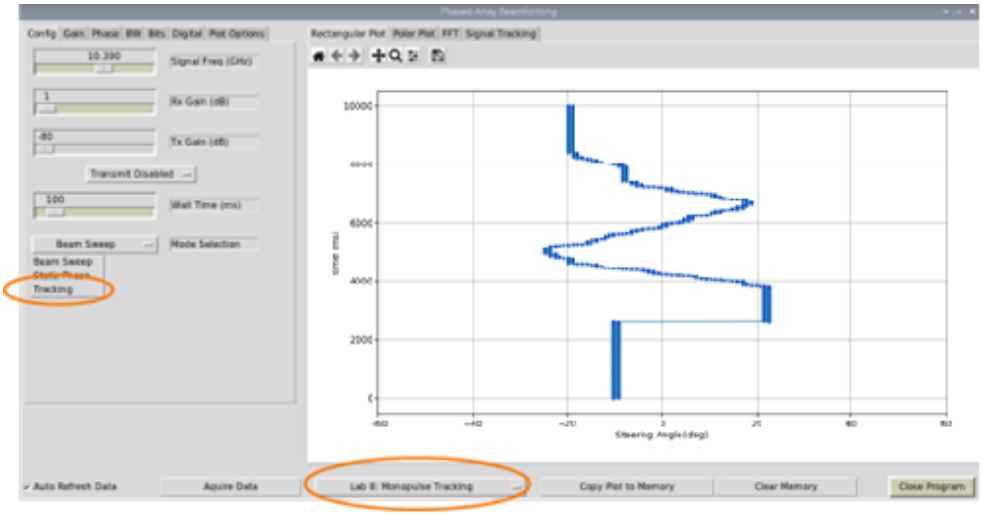
2- Instructions

1- In the Phaser GUI, select "Lab 8: Monopulse Tracking"
2- In the "Gain" tab, select "Blackman" taper



3- From the lecture, what do the "Delta", "Phase Delta", and "Error Function" bars represent?
4- Rotate the array and observe the plots' responses

5- In the "Config" tab, select "Tracking" from the "Mode Selection" pull down menu
6- Move the Phaser antenna and observe the tracking function in action

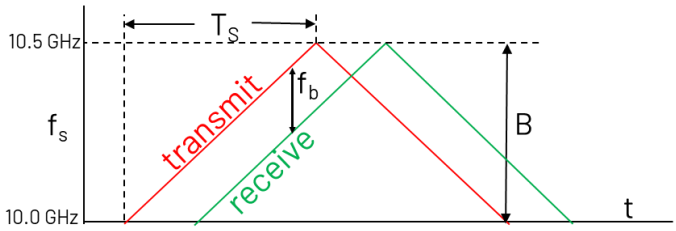


FMCW RADAR

1- Background

In this lab, we will implement the SIMPLEST possible FMCW radar

Recall that an FMCW (frequency modulated continuous wave) radar transmits a continuous frequency ramped signal:



The difference between the transmit and receive frequencies is the "beat frequency", fb. Using this beat frequency, we can compute the range (R) to target as:

$$R = \frac{cT_S}{2B} f_b$$

***c* = speed of light (3E8 m/s)**
***T_s* = transmit frequency ramp rate**
***B* = bandwidth of the frequency ramp**
***f_b* = transmit to receive frequency difference**

In this lab, we use a ramp time of T_S=1 ms and a default frequency ramp bandwidth of B = 500 MHz (though this is adjustable in the lab). With these values, we should expect to see a beat frequency of 3.3 kHz per meter of distance to the target.

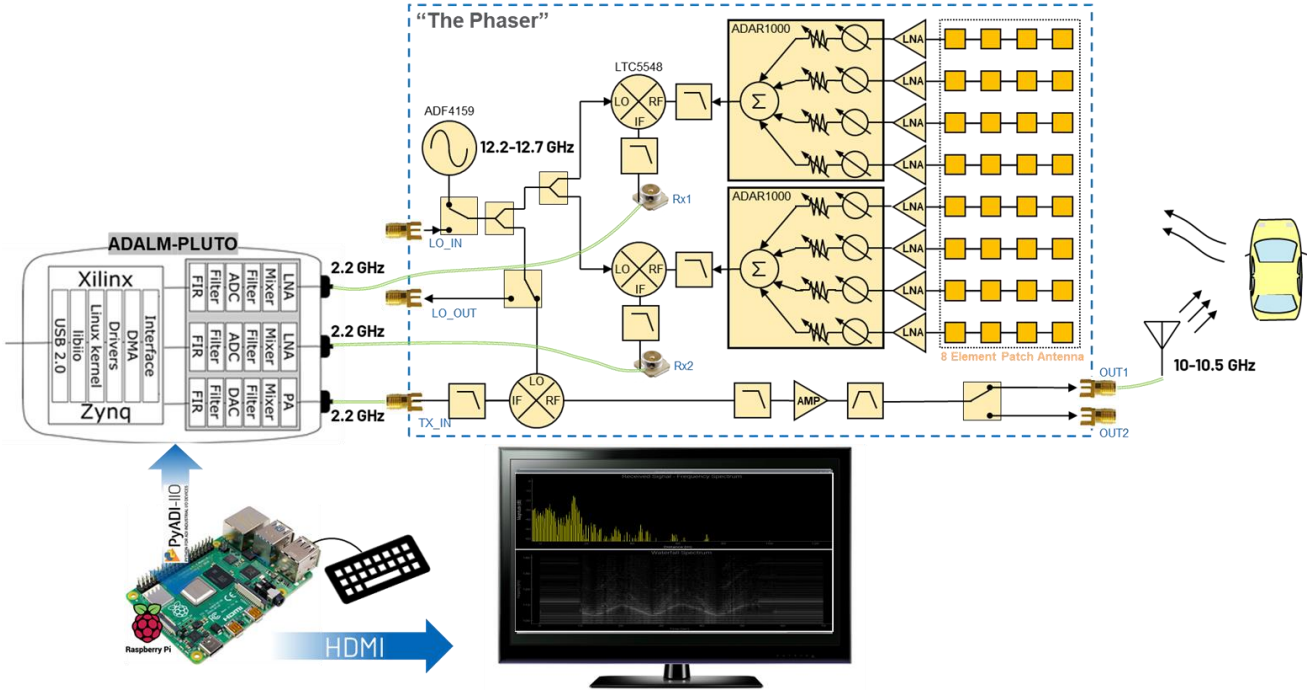
$$f_b = \frac{2 \cdot 1\text{m} \cdot 500\text{MHz}}{3\text{E8m/s} \cdot 1000\mu\text{s}} = 3.3\ kHz \quad per\ 1m\ of\ Range$$

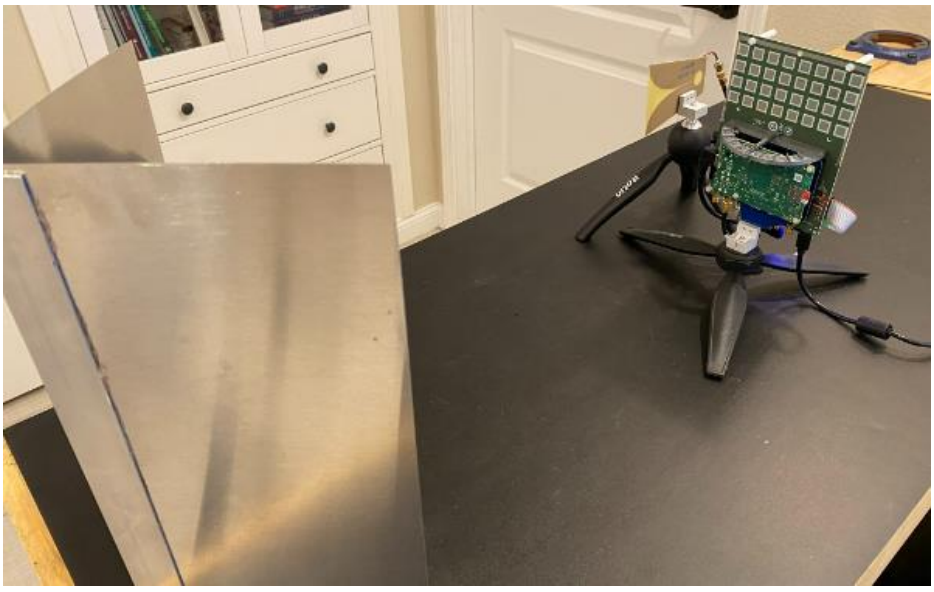
2. Setup

We will transmit a frequency ramped waveform from Phaser and then receive that reflected waveform with the 8 element linear array. That signal will be mixed down, using the LTC5548 on board Phaser. The LO of that mixer is the transmit frequency ramp. Therefore, the mixer result is just the beat frequency. That low bandwidth makes it very easy for Pluto to digitize.

- Ensure that the transmit antenna is connected to "OUT2". This is the SMA port closest to the tripod.
- Ensure that the HB100 is turned off (no LEDs are on)
- Place the corner reflector about a meter away from the receive array

So now our setup looks like this:





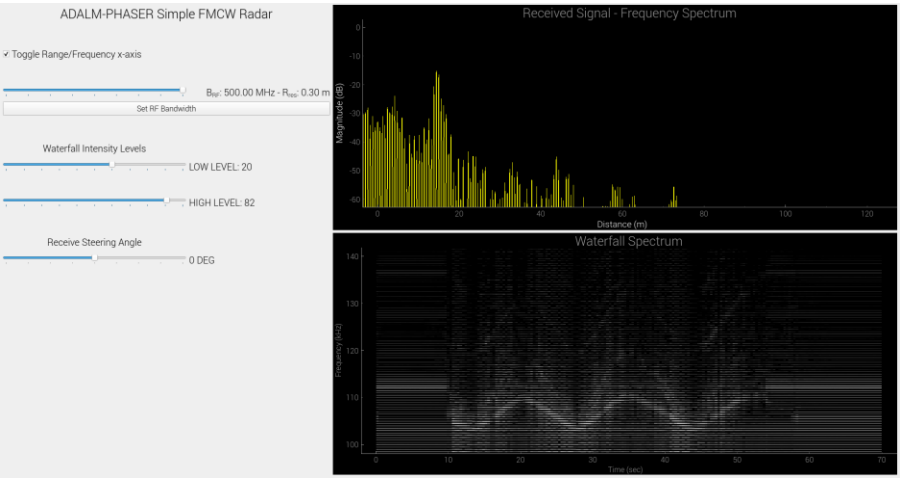
3. FMCW Beat Frequency and Range

14- Open Thonny, by choosing Start → Programming → Thonny

15- Select the “RADAR_FFT_Waterfall.py” tab

16- Click “Run”

17- From Thonny, select “RADAR_FFT_Waterfall.py” and click the green “Run” button



NOTE: If update is VERY slow (i.e. less than 1 update per second), try unplugging the Pluto USB cable, wait 10 sec, plug back in, wait 10 sec, and then click “Run” again.

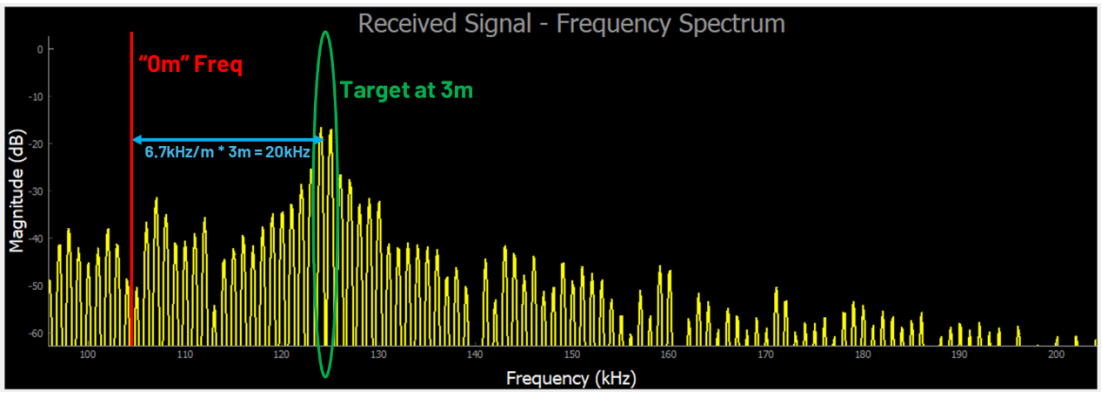
18- The top right graph is the FFT of the beat frequency. The x-axis is frequency, by default. But can be toggled to range by clicking the “Toggle Range” checkbox

19- The bottom right graph is running plot of that FFT graph, over time. Commonly called a “waterfall” plot or spectrogram. The amplitude of the FFT bins is represented by white. The higher the amplitude, the brighter the white.

20- Move the reflective cardboard back and forth in front of the display. Do you see your pattern traced out in the waterfall plot? Try adjusting the LOW and HIGH waterfall intensity levels to eliminate spurious clutter in the plot.

21- Now hold the target very close to the Phaser’s array (i.e. at distance 0m). What is the frequency of the main peak? It won’t be exactly at 100kHz – it might be 103kHz, or 99 kHz, etc. This is the 0m frequency, and is a crude calibration of the system.

22- Now move the target to approximately 1m and observe the FFT plot. Did the frequency move by about 6.7kHz? Try moving to 2m, or 3m.

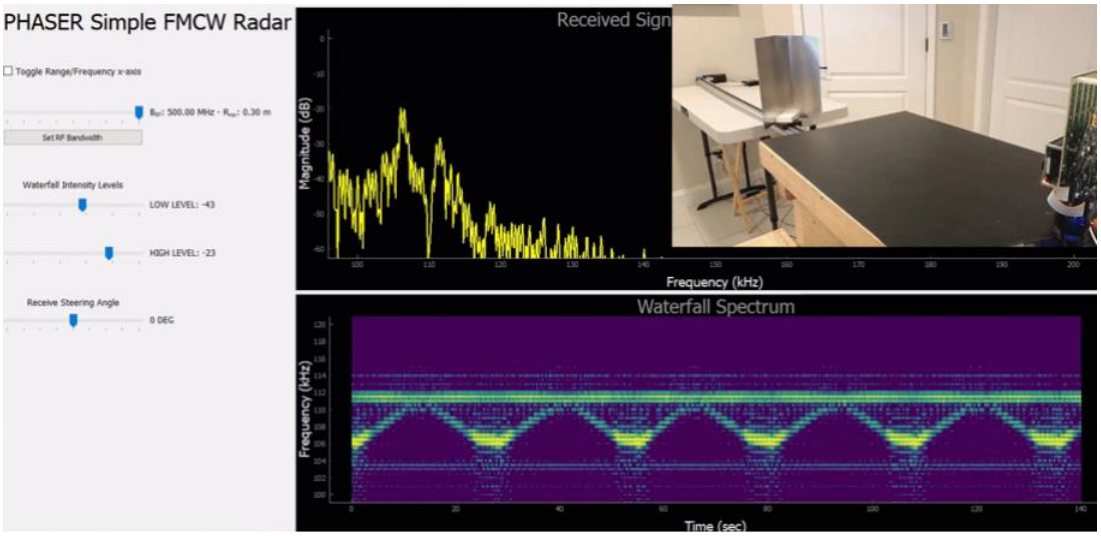


23- With the target still at a fixed distance, try varying the steering angle of the array. Do you notice a change in FFT amplitude as you steer away from the target?

4. Range Waterfall Plot

1- The other graph is running plot of that beat frequency (i.e. range), over time. Commonly called a "waterfall" plot or spectrogram.

2- **SLOWLY and CAREFULLY** (sharp edges!!) move the metal corner reflector back and forth in front of the display. Do you see your pattern traced out in the waterfall plot?



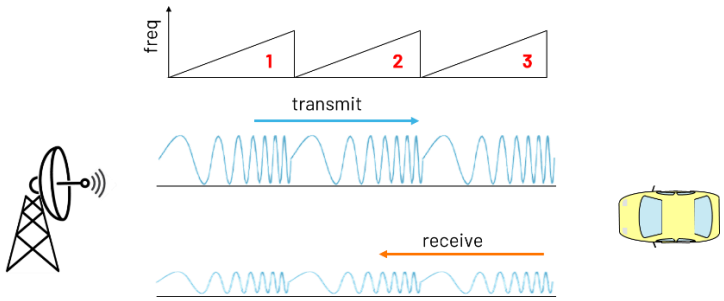
3- Try adjusting the "minIntensity" and "maxIntensity" waterfall levels to eliminate clutter and highlight the return of the target.

RADAR RANGE DOPPLER PLOTTING

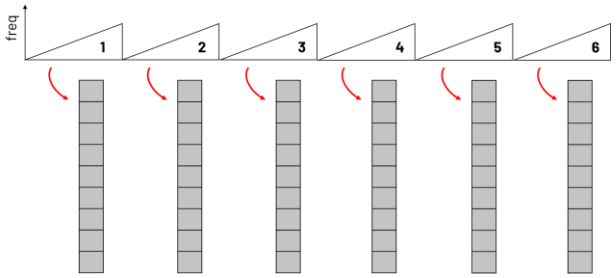
5. Background

In this lab, we'll use multiple chirps to understand how range changes with time (i.e. velocity). It is useful to plot both range and velocity simultaneously in a "Range Doppler" display

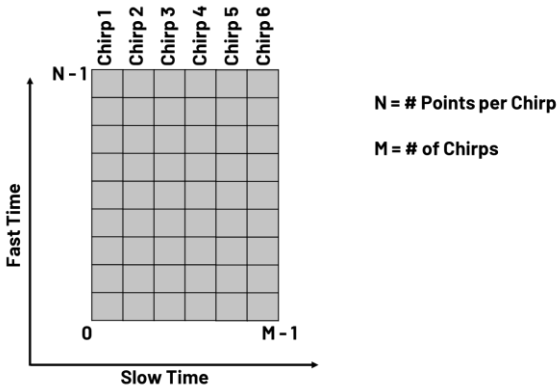
In the previous lab, we saw how to transmit one chirp and get range information from the reflection of that chirp. But things get even more interesting if we send out multiple chirps:



And we can order the data from those chirps into columns:

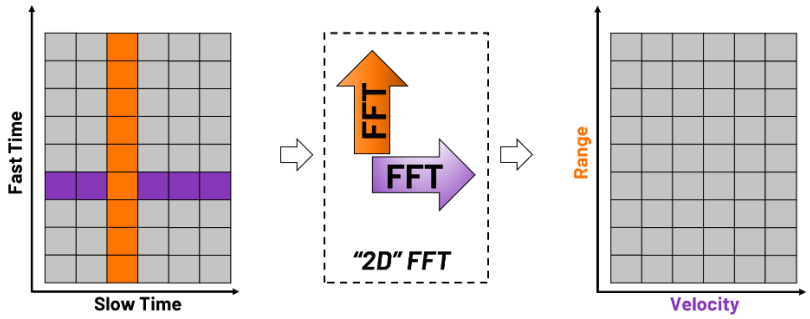


And arrange those chirps into a matrix. The matrix is N rows by M columns. Where N is the number of data points per chirp, and M is the number of chirps.



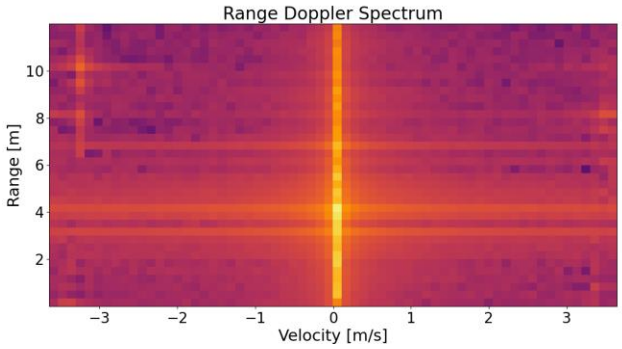
And if you think about it, we have time on the y axis, but we also have time on the x-axis. The time on the y-axis is fast time, because those data points are all separated by the sampling period of the data converter (which is fast!). The x-axis is called slow time if you look across each of the columns, those data points are all separated by the pulse repetition interval –which is on the order of milliseconds

If we then take a 2D FFT of that matrix of slow time and fast time, we will generate a new matrix of Range vs Velocity:



6. Range Doppler Plotting

1- Open the file, "Range_Doppler_Plot.py" and click RUN



2- Wave your hand back and forth and watch the change in target velocity going towards and away from the radar. Also try with a fidget spinner – what does the spinning motion look like on the range doppler plot?

3- Try changing some of the "Key Parameters" in the code – especially the highlighted ones.

```
'''Key Parameters'''
sample_rate = 1e6
center_freq = 1.9e9
signal_freq = 100e3
rx_gain = 30   # must be between -3 and 70
output_freq = 9.8e9
chirp_BW = 500e6
ramp_time = 2000  # us
num_chirps = 64
plot_data = True
save_data = False
f = "phaserRadarData.npy"
```

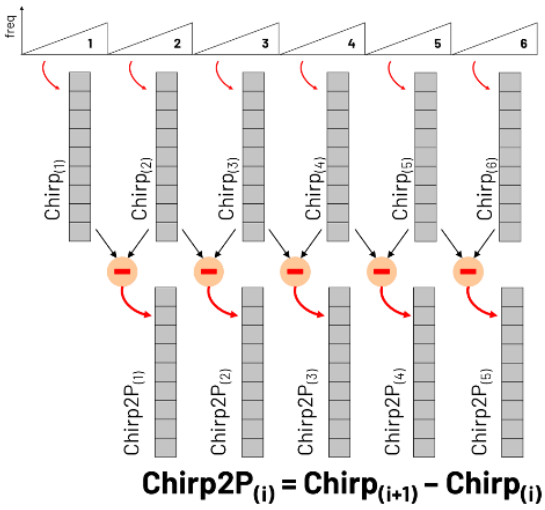
a. Try changing the chirp_BW to 200e6. What happens to the "dot" size of the targets?
b. Try changing the num_chirps to 32 or 128. What happens to the velocity resolution?
c. Try changing ramp time to 500 us.

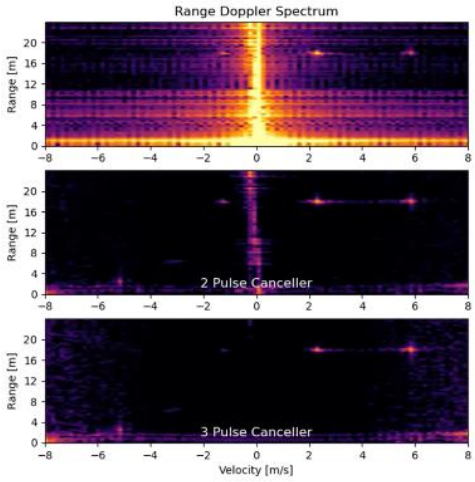
RADAR MTI PROCESSING

7. Background

In this lab, we'll apply MTI pulse cancellation to highlight moving (non zero doppler) targets

MTI stands for moving target indicator. And the goal of MTI radar processing is to enhance the detection of moving targets by eliminating everything that is not moving. In a simple "2 Pulse" MTI canceller, we just subtract the time domain signal from the current pulse from the signal of the previous pulse. If the target is stationary, the signals will be nearly identical, resulting in a difference close to zero. So these targets will be largely cancelled out. If the target is moving, the signals will differ due to the Doppler shift, and so they will not be cancelled out. We do that for all the pulses in the CPI (coherent processing interval).





https://youtu.be/M1eXeqN1c-I

8. Instructions

1- Return to the file: "Range_Doppler_Plot.py". But change "mti_filter" to True

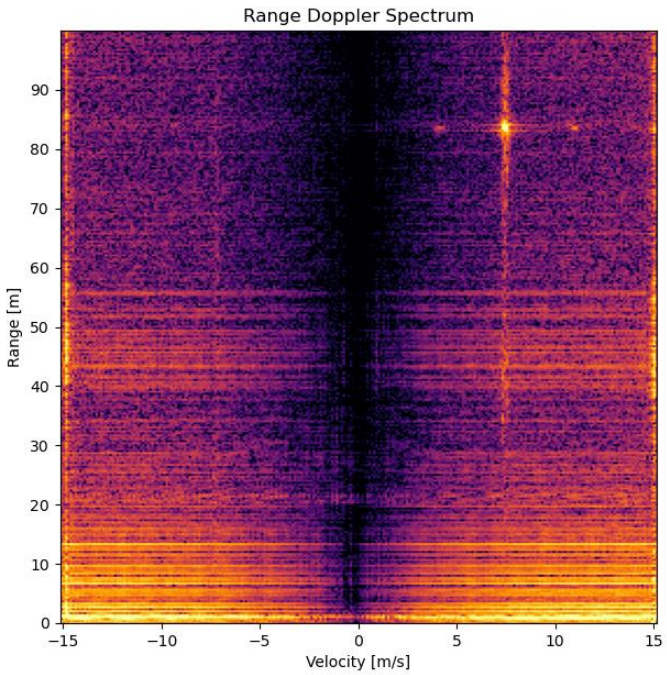
```
69 plot_data = True
70 mti_filter = True
71 save_data = False
```

2- Run the python now and observe the impact of this filter on moving objects – particularly the fidget spinners.
   a. Experiment again with the key parameters above and observe various forms of motion.

3- Open the file “Range_Doppler_Processing.py”
   a. This file plays precorded data of the Phaser tracking a 249g drone from up to 100m away.
   b. You can see the video of the drone’s flight here: https://youtu.be/M1eXeqN1c-I?si=qy5zmvdxZS0KBQCZ&t=441
   c. The parameters this data was taken at are: sample_rate = 4.0 MHz, ramp_time = 300 us, num_chirps = 256 , PRI = 0.5 ms

4- Modify the “MTI_filter” setting on line 50 to: “none”, “2pulse”, and then finally “3pulse”

```
46 f = "DroneFlight.npy"
47 config = np.load(f[:-4]+"_config.npy")
48 all_data = np.load(f)
49
50 MTI_filter = '2pulse'  # choices are none, 2pulse, or 3pulse
51 max_range = 100
52 min_scale = 4   # min intensity level of range doppler plot
53 max_scale = 6   # max intensity level of range doppler plot
```



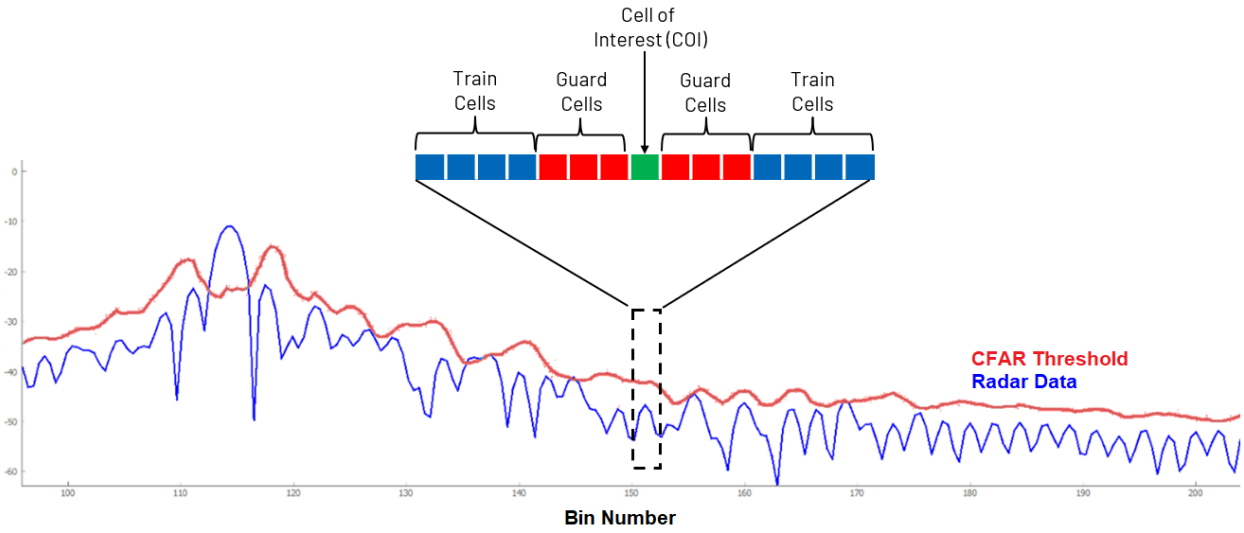
RADAR CFAR PROCESSING

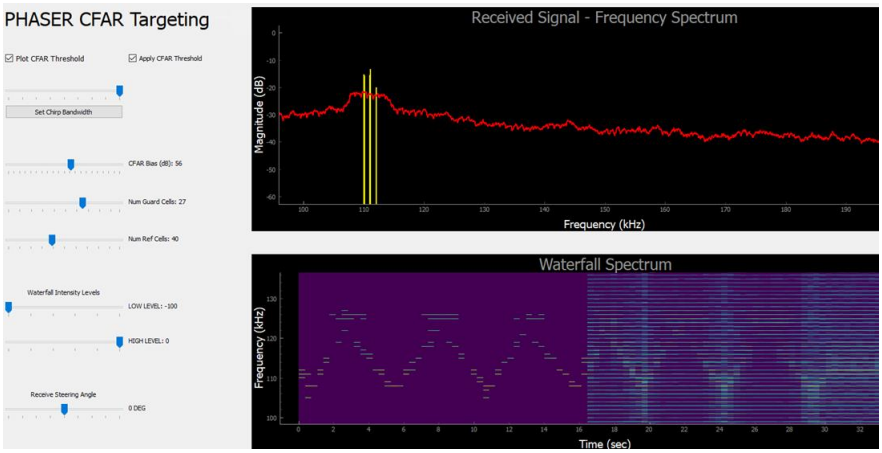
9. Background

In this lab, we'll apply CFAR processing to separate targets from noise

The key parameter in the CFAR algorithm is the probability of false alarm, which remains constant (hence the name). CFAR assumes gaussian noise and looks for outliers in the collected data. The number of standard deviations that the outliers must be considered a detection is based on this probability of false alarm. If the probability of false alarm is set to a higher value, we will see more false alarms but also reduce the likelihood that we miss a true target detection. If the probability of false alarm is set to a lower value, we will see fewer false alarms but also increase the likelihood that we miss a true target detection.

We can adjust the guard and training (reference) cells based on our observation of detection performance.





10. Instructions

1- From Thonny, select "CFAR_RADAR_Waterfall_ChirpSync.py" and click the green "Run" button

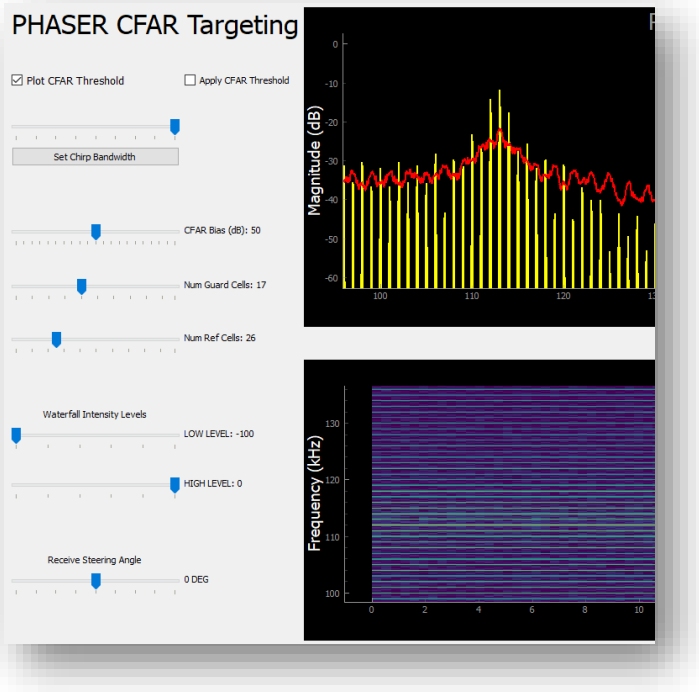
2- Click "Plot CFAR Threshold", but don't click "Apply CFAR Threshold"

3- Adjust "CFAR Bias", "Num Guard Cells", and "Num Ref Cells" so that the red line is below the target peak, but above the clutter peaks

4- Once you have good values, try it out by clicking "Apply CFAR Threshold"

5- Move back and forth (**SLOWLY!**) and see if only your target is now captured.
MOVE SLOWLY!! The Rpi has to process a lot of calculations….

   a. Keep adjusting your values and try to improve! *DON'T "cheat" by changing the Waterfall Intensity Levels!! Leave those at -100 and 0!*



APPENDIX: ADALM-PHASER FREQUENCY PLAN